

OOSC 4.0

AI FOR CIVIC & LEGAL EMPOWERMENT

CIVIC ACTION ENGINE

FINAL STRATEGY • PRODUCT • MVP • ARCHITECTURE • EXECUTION

Three-person team | Antigravity + GitHub

19 August 2026

THE PRODUCT IDEA

One civic AI system that turns a citizen's question or problem into a clear, source-grounded path to action — covering Rights, Schemes, RTI and Forms.

The four illustrative directions in Problem Statement 3 are treated as four capabilities of one coherent product, not four separate applications.

01 | Executive Summary

Selected challenge: Problem Statement 3 — AI for Civic and Legal Empowerment.

Product: Civic Action Engine.

Core promise: Turn a citizen's question, problem, or document into an understandable, source-grounded and actionable civic workflow.

The system should not behave like a generic legal chatbot. It should understand the citizen's goal, identify the relevant civic/legal path, retrieve supporting information, explain it simply, and help prepare the next action.

FOUR CAPABILITY PILLARS

Official direction	Our capability
RTI Drafting Agent	Convert a plain-language information request into a structured, reviewable RTI draft and guide the relevant process where supported.
Rights Navigator	Explain what a citizen may be able to do about a bounded tenant, consumer, workplace or civic/legal situation, with sources.
Scheme Eligibility Reader	Ask relevant questions, compare against supported scheme criteria, explain the result and identify missing evidence.
Conversational Form-Filler	Interview the user for required fields and prepare a reviewable completed form/draft for a supported workflow.

Strategy: Four capabilities. One architecture. One user journey. Build one or two complete demo paths first, while making the remaining capabilities visibly functional within the same system.

02 | Official Problem Statement

AI for Civic and Legal Empowerment

Theme: Civic Tech, Legal Access and Government Transparency

Problem Context

Citizens often have legitimate rights and entitlements — consumer protections, tenant rights, RTI access, welfare eligibility — that go unused because navigating bureaucratic and legal language is intimidating and time-consuming. Relevant information is scattered across PDFs, notices and portals that are not designed to solve a specific personal problem quickly.

Challenge Statement

Build an AI system that helps a citizen understand and act on their civic or legal rights, translating bureaucratic complexity into a clear, guided path.

Illustrative Directions

- **RTI Drafting Agent** — converts a plain-language question into a properly formatted application to the right department.
- **Rights Navigator** — explains, in simple terms, what a person can do about a specific tenant, consumer or workplace dispute.
- **Scheme Eligibility Reader** — reads government portals and answers eligibility questions in plain language.
- **Conversational Form-Filler** — interviews the user and auto-populates the official form.

- **Open reinterpretation** of bureaucracy translation is welcome.

Source: supplied OOSC 4.0 Hackathon Problem Statements PDF, Problem Statement 3. The visible citation markers have intentionally been removed from this presentation copy.

03 | The Real Problem

The brief frames the gap as information being scattered across PDFs, notices and portals and not being designed to solve a person's specific problem quickly.

OUR PROBLEM MODEL

CITIZEN PROBLEM → SCATTERED INFORMATION → COMPREHENSION GAP → DECISION GAP → ACTION GAP

Layer	Citizen asks	Product response
Understand	"What does this mean?"	Simple explanation tied to source material.
Determine	"Does this apply to me?"	Relevant questions + grounded issue/eligibility reasoning.
Act	"What do I do now?"	Ordered steps + evidence/document checklist.
Complete	"What should I submit?"	Reviewable RTI/form/complaint/action document.

04 | Product Architecture

The four illustrative directions become modules inside one product rather than four disconnected applications.

Module	Input	AI job	Output
Rights Navigator	Situation / optional document	Classify + retrieve + explain + guide	Rights/process explanation + next steps
Scheme Eligibility Reader	Profile + scheme question	Extract criteria + compare + explain	Potential eligibility + missing evidence
RTI Drafting Agent	Plain-language information request	Structure request + guide process	Reviewable RTI draft
Conversational Form-Filler	Conversation + required fields	Ask missing questions + validate	Reviewable completed draft/form

Shared services: source retrieval • user/session state • document extraction • structured AI output • source display • document generation

05 | Core User Flow

- 1 **Enter:** citizen describes a problem, question, document or goal.
- 2 **Route:** system identifies Rights, Scheme, RTI or Form path.
- 3 **Clarify:** ask only required follow-up questions.
- 4 **Retrieve:** fetch relevant authoritative/source material.
- 5 **Guide:** produce a structured explanation and action plan.
- 6 **Complete:** list evidence and generate a reviewable draft when supported.
- 7 **Verify:** show sources and clearly label uncertainty.

06 | Example Scenarios

A — RTI: “I want to know how much money was allocated for road construction in my locality.” → clarify → RTI Drafting Agent → structured draft → review → process/source guidance.

B — Scheme: “Am I eligible for this scheme?” → Scheme Eligibility Reader → questions → criteria comparison → reasons + missing documents → Form-Filler if appropriate.

C — Rights: citizen describes a tenant/consumer/workplace issue → Rights Navigator → issue understanding → source retrieval → explanation → action checklist.

D — Form: citizen cannot understand a form → Conversational Form-Filler → conversational interview → field validation → reviewable completed draft.

07 | MVP Scope

Priority	Feature	Decision
P0	Unified intake + routing	Must build
P0	Rights Navigator	Must build
P0	RTI Drafting Agent	Must build
P0	One controlled Scheme Eligibility workflow	Must build
P0	One controlled Conversational Form-Filler workflow	Must build
P0	Source/evidence display	Must build
P0	One document-upload path	Recommended
P1	Voice + multilingual input	Only after core stability
P1	More domains/schemes/forms	Expand only if time allows
P2	Long-term case management	Roadmap

Scope rule: “All four directions” means all four capabilities are represented — not that we support every Indian law, scheme and government form. Each is demonstrated with a controlled, defensible workflow.

08 | Technical Architecture

Layer	Technology	Role
Frontend	React / Next.js + Tailwind CSS	Responsive citizen workflow, module screens, upload, results and sources.
Backend	Node.js / Express or Next.js API routes	Authentication, validation, orchestration and APIs.
AI	LLM API + structured prompts	Classification, clarification, retrieval-grounded reasoning and drafting.
Retrieval	PostgreSQL / Supabase + vector search	Store and retrieve approved source content.
Documents	PDF/text extraction + OCR where required	Read supported notices/forms/documents.

Layer	Technology	Role
Deployment	Vercel + managed services	Public demo hosting.
Version control	GitHub + feature branches	Parallel work, reviews and stable main branch.

Flow: WEB APP → API → VALIDATION → ROUTER → RETRIEVAL / DOCUMENT PROCESSING → LLM → STRUCTURED OUTPUT → SOURCE CHECK → DATABASE → UI

09 | Trust, Accuracy & Safety

- **Do not present the product as a lawyer.** It is a civic/legal information and navigation assistant.
- **Ground outputs.** Prefer official government/legal sources for supported workflows.
- **Show sources.** Judges and users should see where important guidance came from.
- **Separate fact from inference.**
- **Review before submission.** Generated documents remain editable and user-approved.
- **Ask when uncertain.** Never invent jurisdiction, eligibility or procedure.
- **Minimise sensitive data.** Store only what is needed.
- **Graceful failure.** If AI/retrieval fails, explain the failure instead of producing unsupported instructions.

10 | Three-Person Team

Member	Primary ownership	Secondary ownership
Member 1 — Tech Lead	Backend, AI integration, DB, GitHub, deployment, final integration	Architecture, debugging, release management
Member 2 — Frontend	React/Next UI, responsive design, module screens, API integration	Visual polish + demo flow
Member 3 — AI / Research / QA	Sources, prompts, retrieval testing, evaluation	QA, edge cases, docs and demo support

Antigravity rule: accelerate coding with AI, but keep architecture, interfaces, environment variables, source provenance and Git history understandable. Test generated code before merging.

11 | GitHub Repository Plan

Structure

```
project-root/
├── frontend/
├── backend/
├── ai/
├── knowledge-base/
├── docs/
├── tests/
├── .env.example
├── README.md
└── LICENSE
```

Branches

Branch	Purpose
main	Stable, demo-ready code only.
feature/frontend	Frontend implementation and UI.
feature/backend	API, database and integration.
feature/ai	Prompts, retrieval, knowledge base and evaluation.

Workflow: Pull main → work on branch → test → commit → push → review/merge → pull main.

12 | Execution Plan

Date	Focus	Exit condition
19 Aug	Lock concept, four modules, source set, repo and API contracts.	Everyone knows exactly what is being built.
20 Aug	Parallel frontend, backend and AI/retrieval implementation.	Components work independently.
21 Aug	Integrate all four module paths and controlled demo scenarios.	End-to-end happy path works.
22 Aug	Deploy, test, polish UI, README and demo preparation.	Hosted prototype stable; feature freeze.
23 Aug	Final QA, video, submission checks and buffer.	Submission-ready.

Feature freeze: 22 August night. No major functionality on submission day.

13 | Demo Strategy

Time	Demo moment	Judge takeaway
0:00–0:30	Citizen pain + fragmented information.	Real problem.
0:30–1:00	Introduce Civic Action Engine.	Clear solution.
1:00–2:00	Rights Navigator scenario.	Understands issue.
2:00–2:45	RTI Drafting Agent.	Turns intent into action.
2:45–3:30	Scheme Eligibility Reader.	Personalises public information.
3:30–4:15	Conversational Form-Filler.	Reduces bureaucratic friction.
4:15–5:00	Sources + architecture + AI pipeline.	Technically grounded.
5:00–5:30	Impact + roadmap.	Scalable product.

Do not demo four unrelated pages. Show that one engine routes the citizen to the appropriate capability.

14 | What We Must NOT Do

- Do not build a generic “Ask AI anything about law” chatbot.
- Do not attempt every Indian law, scheme and government form.
- Do not rely on the LLM's memory for legal/procedural facts.
- Do not add voice, multilingual support, dashboards and case tracking before the core workflow works.

- Do not automatically submit generated documents without user review.
- Do not sacrifice a stable demo for a flashy feature.

Golden rule: Build less, but make the important path excellent.

15 | Final Product Definition

ONE-LINE PITCH

“Civic Action Engine helps citizens turn confusing civic and legal problems into clear, source-grounded actions — from understanding rights and checking scheme eligibility to drafting RTIs and completing forms.”

Core differentiator: We are not merely translating bureaucracy. We are building an **action layer** connecting understanding to completion.

01	RIGHTS	Understand what applies and what to do.
02	SCHEMES	Understand eligibility and missing requirements.
	RTI	Turn an information need into a structured RTI draft.
04	FORMS	Turn a conversation into a reviewable form/draft.

16 | Immediate Next Steps

- 1 **Freeze** the four-module architecture.
- 2 **Select** one controlled demo scenario for each module.
- 3 **Collect** authoritative source material for those scenarios.
- 4 **Create** the GitHub repository and branches.
- 5 **Scaffold** frontend/backend/AI in Antigravity.
- 6 **Define** API contracts before parallel coding.
- 7 **Build** Rights + RTI first.
- 8 **Add** one scheme workflow and one form workflow.
- 9 **Deploy and test** before optional features.

First milestone: a judge can open the deployed website and complete at least one meaningful civic task end-to-end.

17 | Sources & Evidence

Primary challenge source: supplied OOSC 4.0 Hackathon Problem Statements PDF — Problem Statement 3, including its context, challenge statement and four illustrative directions.

Important distinction: The four illustrative directions are source-derived. The decision to unify them into Civic Action Engine, the MVP, architecture, team split, timeline and recommendations are our strategic interpretation.